# INTERNSHIP PROJECT REPORT

# TASK 4

"Computer Vision Object Detection and Frame-by-Frame Segmentation Pipeline"

**Progree AI Internship Program**

By

**Name:** Shazmeen Irshad

**Student ID:** B9/2929

# 1. Abstract:

This project focuses on developing a computer vision pipeline for processing video frames using Python and OpenCV. The main objective is to extract contour-based regions from a video, apply image preprocessing techniques, and display dynamic information on processed frames. The pipeline uses Gaussian Blur to reduce image noise, grayscale conversion to simplify image representation, and Adaptive Thresholding to separate foreground regions from the background. Contours are then extracted from the processed frames and filtered according to their pixel area. The system processes video frame by frame, draws detected contours, and overlays frame numbers and contour counts. Performance is evaluated using frame-level contour statistics, processing time, and average processing FPS. The pipeline processed a total of **1,507 frames**, with an average of **23.67 contours** per frame and an average processing time of **0.1649 seconds** per frame. This project demonstrates the practical application of image preprocessing, contour extraction, and video analysis using OpenCV.

# 2. Introduction:

Computer Vision is a field of Artificial Intelligence that enables computers to analyze and understand visual information from images and videos. It is commonly used in surveillance, object detection, traffic monitoring, medical imaging, robotics, and automated inspection. In this project, a video processing pipeline was developed using OpenCV. Instead of using a deep learning model such as YOLO, the project uses traditional computer vision techniques to identify contour regions within video frames. The pipeline processes each frame independently and applies several image processing operations. These operations help simplify the image and identify regions that may represent objects or shapes. The detected contours are filtered using pixel-area boundaries to remove very small regions and unusually large regions. The remaining contours are drawn on the original frame, and the system displays the frame number and contour count. This project provides practical experience with video processing, image segmentation, contour extraction, and performance evaluation.

# 3. Problem Statement:

Processing video manually is time-consuming and difficult when a large number of frames need to be analyzed. The goal of this project is to create an automated pipeline that can process video frames, extract contour regions, and display useful information for every frame.

The system should:

- Process video frames automatically.
- Reduce image noise using Gaussian Blur.
- Apply Adaptive Thresholding.
- Extract contours from processed frames.
- Filter contours based on pixel area.
- Display contour information on video frames.
- Record processing statistics and performance metrics.

# 4. Project Objectives:

The main objectives of the project are:

1. Build a video processing pipeline using OpenCV.
2. Apply Gaussian filtering for image smoothing.
3. Convert video frames into grayscale.
4. Apply Adaptive Thresholding for segmentation.
5. Extract contours from thresholded images.
6. Filter contour regions based on pixel-area limits.
7. Draw detected contours on video frames.
8. Overlay dynamic frame numbers and contour counts.
9. Evaluate frame-by-frame processing performance.
10. Export logs, graphs, and performance summaries.

# 5. Tools and Technologies:

1. **Python:**
   Python was used as the primary programming language due to its simplicity and extensive support for computer vision and image processing.
2. **OpenCV:**
   OpenCV is an open-source computer vision library used for video processing and image analysis. It was used to read video frames, apply image filters, perform thresholding, extract contours, draw shapes, and save processed video output.
3. **NumPy:**
   NumPy was used for numerical operations and handling image data represented as arrays of pixel values.

4. **Matplotlib:**
   Matplotlib was used to create graphs for contour counts and processing time measurements.
5. **Google Colab:**
   Google Colab was used as the development environment for executing Python code and processing the uploaded video.

# 6. Methodology:

The proposed pipeline consists of multiple image processing stages. Each video frame is processed independently through preprocessing, segmentation, contour extraction, filtering, and visualization.

1. **Video Input and Frame Extraction:**
   The input video was loaded using OpenCV's **VideoCapture** function.

```
Saving 14737069_3840_2160_60fps.mp4 to 14737069_3840_2160_60fps.mp4
```

   The video properties were inspected, including frame count, resolution, and frame rate. The video contained a total of **1,507 frames**. Each frame was extracted and passed through the image processing pipeline.

```
FPS: 60.0
Total Frames: 1507
Resolution: 3840 x 2160
```




## 2. Gaussian Blur:

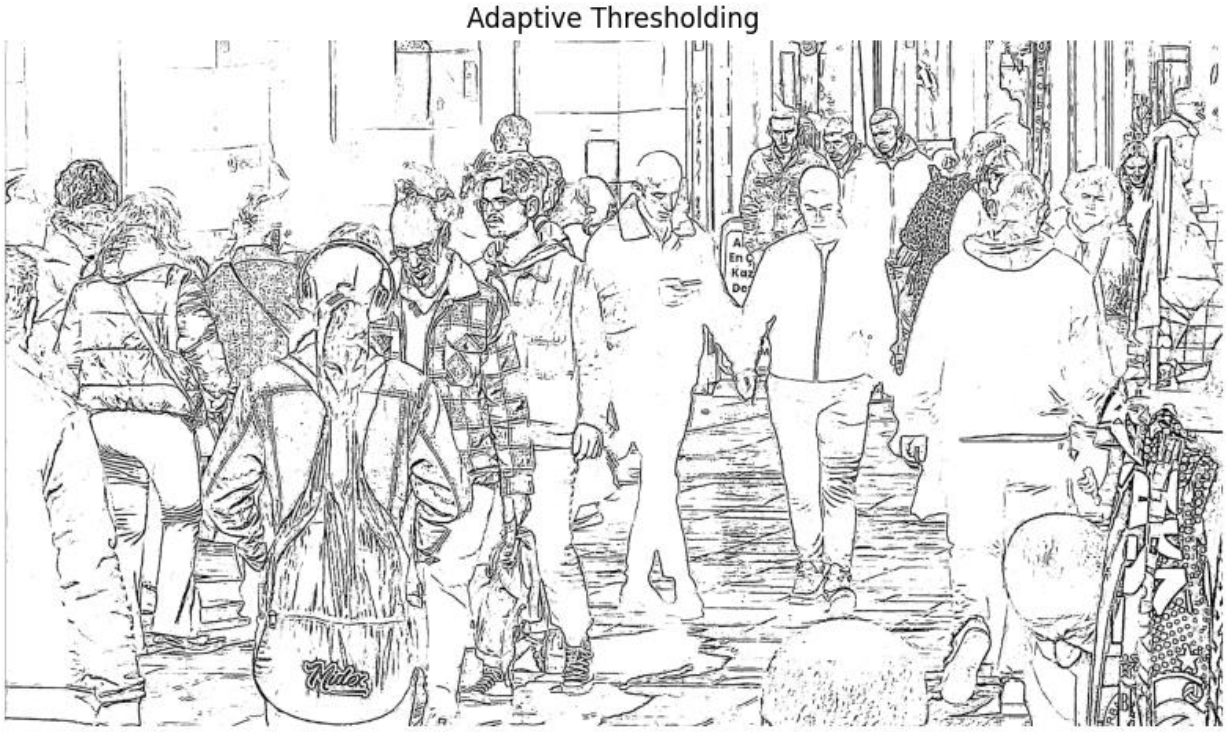
Gaussian Blur was applied to reduce image noise and small variations in pixel intensity. Noise can produce unwanted edges and contours during image segmentation. Applying a Gaussian filter smooths the image and helps reduce some unwanted regions. The project used a Gaussian kernel size of **5 × 5.** Gaussian filtering performs weighted averaging using neighboring pixel values. The operation helps prepare the image for subsequent processing.

Gaussian Blur



### 3. Grayscale Conversion:

The blurred frame was converted into grayscale. A color image contains multiple channels, whereas a grayscale image represents intensity using a single channel. Grayscale conversion simplifies the image and makes threshold-based segmentation easier. The resulting grayscale image was used as input for Adaptive Thresholding.



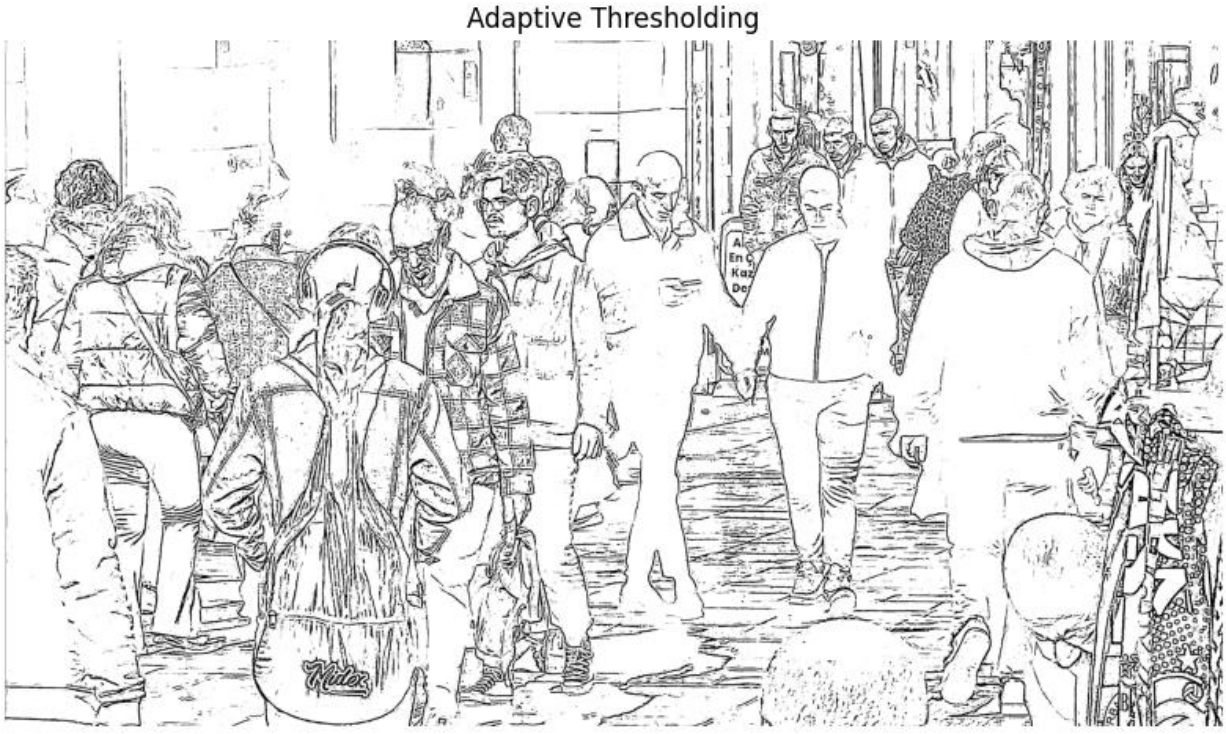

## 4. Adaptive Thresholding:

Adaptive Thresholding was used to convert the grayscale image into a binary image. Unlike global thresholding, Adaptive Thresholding calculates threshold values based on local regions of the image. This approach can be useful when lighting conditions vary across the frame.

The following parameters were used:

- Thresholding method: **ADAPTIVE_THRESH_GAUSSIAN_C**
- Threshold type: **THRESH_BINARY**
- Block size: **11**
- Constant C: **2**

The output binary image separates regions based on their intensity values and provides the input for contour extraction.




## 5. Contour Extraction:

Contours represent the boundaries of connected regions in an image. The **cv2.findContours()** function was used to extract contours from the binary image.

The following settings were used:

- Retrieval mode: **RETR_EXTERNAL**
- Approximation method: **CHAIN_APPROX_SIMPLE**

  The **RETR_EXTERNAL** method retrieves external contours, while **CHAIN_APPROX_SIMPLE** reduces the number of stored boundary points. The extracted contours represent image regions and do not necessarily correspond to individual real-world objects.

## 6. Contour Filtering:

The extracted contours were filtered according to their pixel area to reduce unwanted detections.

The project used the following area boundaries:

- Minimum contour area: 500 pixels
- Maximum contour area: 50,000 pixels

Contours smaller than the minimum area were excluded. Contours larger than the maximum area were also removed. This step helps reduce small noise regions and excessively large regions. The remaining contours were drawn on the video frames.

```
Total contours detected: 364
```

```
Filtered objects: 27
```

## 7. Dynamic Frame Overlay:

The pipeline adds dynamic text information to the processed frames.
The overlay includes:

- Current frame number
- Number of filtered contours detected in the frame

This information is displayed using OpenCV text drawing functions. The overlay allows users to observe frame progression and changes in contour counts throughout the video

```
Processed frames: 1507
Total detected contour objects: 35674
```

Processed Frame with Object Count



.

## 8. Video Processing and Output:

The complete video was processed frame by frame. For every frame, the pipeline performed the following operations:

1. Read the video frame.
2. Apply Gaussian Blur.
3. Convert the frame to grayscale.
4. Apply Adaptive Thresholding.
5. Extract contours.
6. Filter contours by pixel area.
7. Draw contours on the frame.
8. Add frame number and contour count.
9. Save the processed frame.
10. Record processing information.

The processed video was saved as:

```
Processed video saved: processed_output.mp4
Total frames processed: 1507
```

# 7. Performance Evaluation:

Performance evaluation was conducted to measure the processing behavior and efficiency of the pipeline. The evaluation included total frames processed, contour count statistics, processing time, and average processing FPS.

## 1. Performance Metrics:

The following metrics were recorded:

- Total number of frames processed
- Average contour count per frame
- Maximum contour count
- Minimum contour count
- Average processing time per frame
- Average processing FPS

## 2. Performance Results:

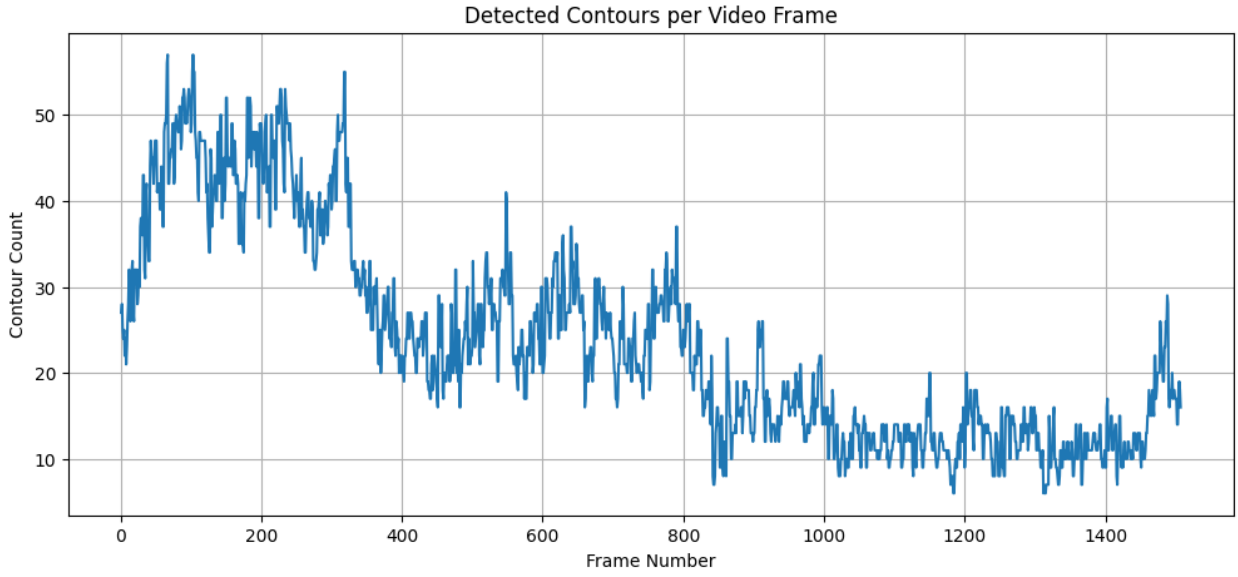
The following results were obtained from the notebook:

```
===== TASK 4 PERFORMANCE SUMMARY =====
Video: 14737069_3840_2160_60fps.mp4
Total Frames: 1507
Average Contours per Frame: 23.67
Maximum Contours: 57
Minimum Contours: 6
Average Processing Time: 0.1649 seconds
Average Processing FPS: 6.07
```

The pipeline processed **1,507 frames** and detected an average of **23.67 contour** regions per frame. The maximum number of filtered contours detected in a frame was **57**, while the minimum was **6**. The average processing time was **0.1649 seconds per frame**, corresponding to approximately **6.07 FPS**. These results describe the performance of the implemented pipeline in the execution environment used for the project.

## 3. Contour Count Per Frame:

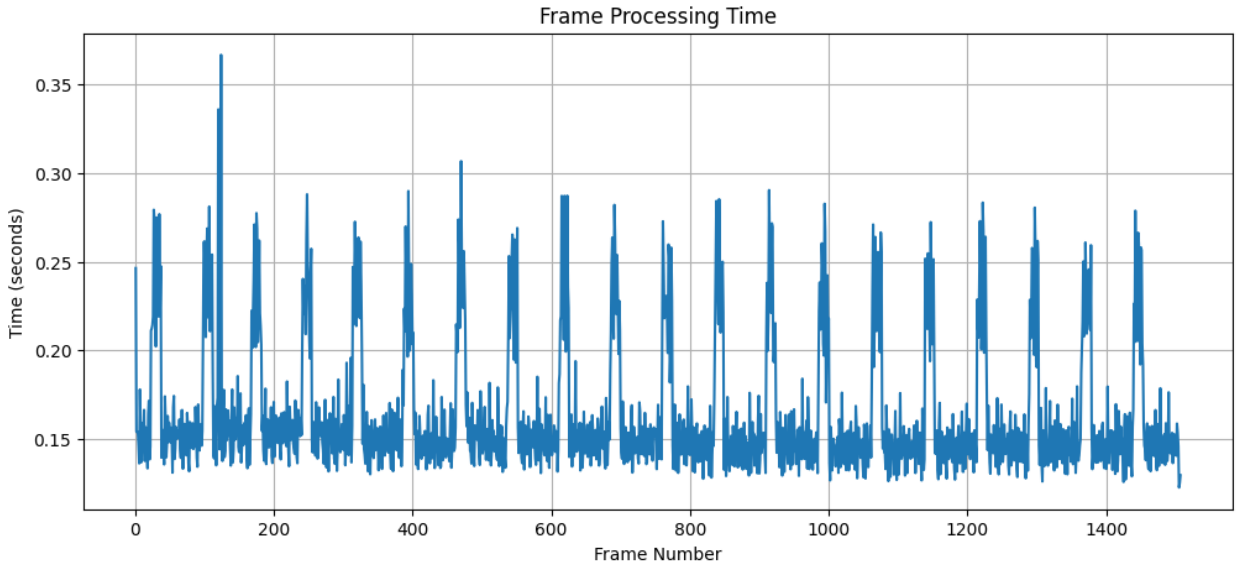
A graph was generated to visualize the number of detected contour regions in each frame. The contour count varied throughout the video due to changes in image content, lighting, background, and visible shapes. The graph helps identify frames containing higher or lower numbers of detected contour regions.




## 4. Processing Time Per Frame:

Processing time was recorded to evaluate the time required to process individual frames. A graph was generated to visualize processing time across the video. Processing time may vary due to image complexity, contour count, and computational workload. The

measured average processing time was **0.1649 seconds per frame**, with an average processing speed of approximately **6.07 FPS**. This processing speed is below the input video's original frame rate of **60 FPS**. Therefore, the current implementation is considered a video processing pipeline and does not guarantee real-time processing at the original frame rate.




### 5. Exported Logs:

The following CSV files were generated:

- **frame_detection_log.csv**
- **processing_time_log.csv**
- **performance_summary.csv**

These files preserve contour counts, processing time measurements, and summary statistics for further analysis.

## 8. Discussion:

The project demonstrates the practical use of traditional computer vision techniques for video processing. Gaussian Blur helps reduce noise before segmentation. Grayscale conversion simplifies the image representation, while Adaptive Thresholding separates regions based on local intensity values. Contour extraction identifies boundaries of connected regions in the binary image. Area-based filtering removes some small and excessively large regions. The dynamic overlay provides frame-level information, while the performance graphs show variations in contour counts and processing time. The pipeline successfully processes video frames and produces visual output and performance logs.

# 9. Limitations:

- **Contour-Based Detection:**
  The system identifies contour regions rather than recognizing object categories. It cannot determine whether a contour represents a person, car, animal, or another object.

- **No Semantic Classification:**
  The pipeline does not use a trained deep learning model and does not assign semantic labels to detected regions.

- **Contour Count Is Not Unique Object Count:**
  A single physical object may generate multiple contours, while multiple objects may appear connected as one contour. Therefore, contour count should not be interpreted as an exact count of unique physical objects.

- **Sensitivity to Image Conditions:**
  The results may vary depending on lighting, background complexity, image contrast, and thresholding parameters.

- **Processing Speed:**
  The average processing speed was approximately **6.07 FPS**, which is lower than the input video's original frame rate of **60 FPS**.

- **No Persistent Object Tracking:**
  The system processes frames independently and does not assign persistent IDs or track objects across multiple frames.

# 10. Future Improvements:

The current pipeline can be improved through the following approaches.

- **Deep Learning-Based Object Detection:**
  A pretrained YOLO model could be integrated to identify specific object classes such as people, cars, bicycles, and other objects. This would provide semantic object detection instead of contour-only detection.

- **Object Tracking:**
  Algorithms such as SORT or DeepSORT could be integrated to assign IDs to detected objects and track them across frames.

- **Parameter Optimization:**
  The Gaussian kernel size, Adaptive Thresholding block size, constant C, and contour area boundaries could be optimized for different video conditions.

- **Performance Optimization:**
  Processing speed could be improved by resizing frames, using optimized OpenCV operations, processing fewer frames when appropriate, or using GPU acceleration where supported.

- **Improved Segmentation:**
  More advanced segmentation methods could be used to improve performance under complex lighting and background conditions.

# 11. Conclusion:

This project successfully implemented a computer vision pipeline for video processing using Python and OpenCV. The pipeline applies Gaussian Blur, grayscale conversion, Adaptive Thresholding, contour extraction, and area-based filtering to identify contour regions in video frames. Dynamic overlays display frame numbers and contour counts, while performance logs and graphs provide information about processing behavior. The **pipeline processed 1,507 frames**, detected an average of **23.67 contour regions per frame**, and achieved an average processing speed of approximately **6.07 FPS**. The project demonstrates the practical application of traditional computer vision techniques and provides a foundation for more advanced systems involving deep learning-based object detection and tracking. Although the current implementation does not perform semantic object recognition, it successfully demonstrates contour-based video processing and segmentation.

.